Lab session 4-5

Elias Deruytter

November 2023

1 Introduction

In this lab I was asked to take the dot product of two matrices. To start, this was done using threads so that the multiplication of the different columns and rows can run in parallel. With this implementation, a full column and a row must be fetched from global memory for each thread, or for each element of the resulting array. This is extremely time consuming.

So The first implementation did not use shared memory, which makes the process slow. In the second implementation, shared memory was also used. Each block has a shared memory, which all threads within a block share among themselves. This is used to fetch all (possible) data from global memory to shared memory at the beginning of each thread and then continue with shared memory in the thread code.

2 Implementation

The amount of threads available in a block is 1024, this means only 1/4th of my 64 by 64 matrix can be calculated in each block. In my example only 2 matrices (2 = k factor) of 32 by 32, fit inside the shared memory. It was not checked whether more elements can be stored in shared memory. For example, if 4 '32 by 32' matrices would fit, then two iterations (2 for loops) are no longer necessary within the thread, this would be an extra optimisation. However, this is not completely true. To extract the 2 smaller matrices in matrix A form shared memory, 2 iterations are still required, because there are only 1024 threads available to store the elements in shared memory. The same would apply for dot products with bigger k factors.

So to implement this faster method, 4 blocks where used with a shared mem-

ory containing 2 smaller matrices of 32*32. These two smaller matrices where called smallerBlockA and smallerBlockB. To illustrate this (figure 1): The two yellow 32 by 32 matrices are now smallerBlockA and smallerBlockB, the first iteration is done with these two in shared memory. Every thread in the block can now calculate it's corresponding answer. The answer is a multiplication of a row of matrix A with a column of matrix B (shown as dark blue rectangles in the figure). This process is repeated for the red matrices (figure 1). Again, the red matrices are transferred to shared memory to calculate the answers for each element (= for each thread). The two answers in the two iterations are now summed up to arrive at the blue matrix. This process does the same as a normal dot product, but in two steps.

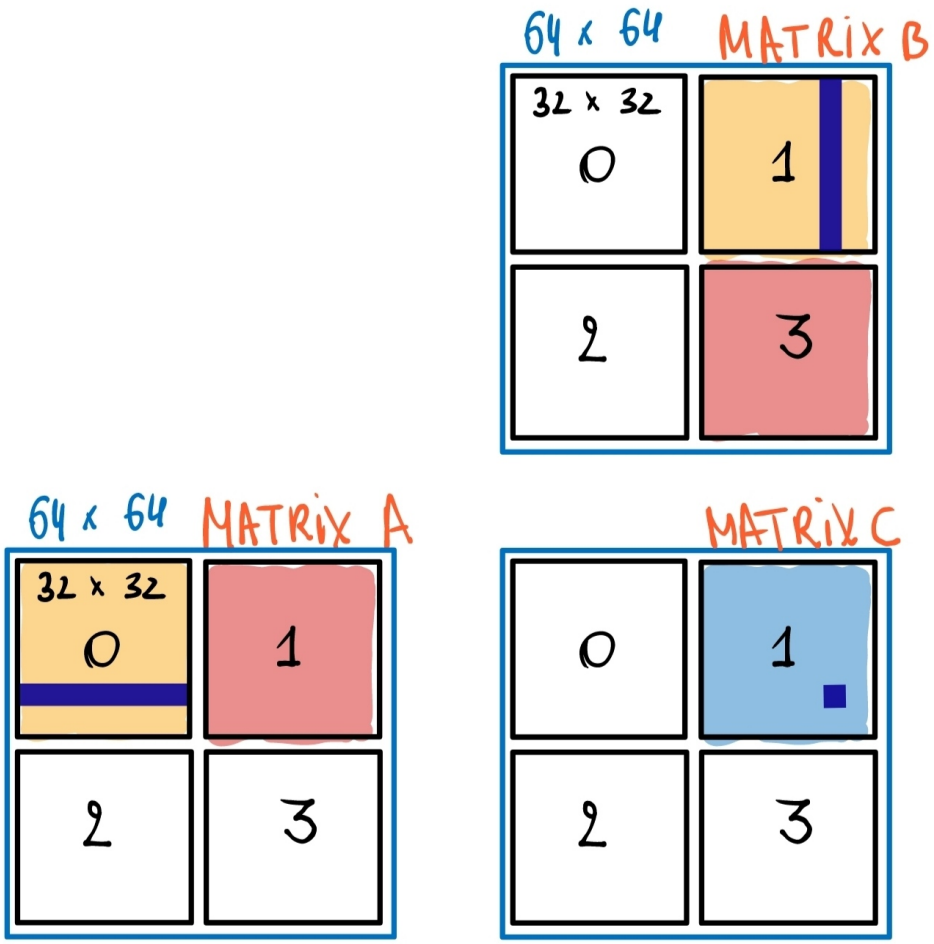


Figure 1: Illustration

3 Timing

As can be seen in figure 3 and 2, the timing of the implementation with shared memory is almost 4 times better. This is of course because the threads don't have to fetch from global memory in every iteration inside the thread, but instead fetches from shared memory, which is faster.

```
-------------------------timeDelta without shared memory: ------------------------
411
```

Figure 2: without shared memory

```
-------------------------timedelta with shared memory, k factor 2: ------------------------
129
```

Figure 3: With shared memory